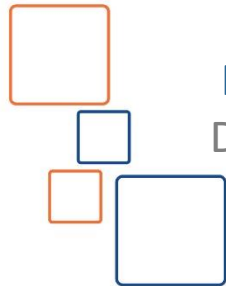


CORE JAVA PROGRAMMING

AN INTRODUCTION TO JAVA



Presented By
Dr. Eman Hesham, DBA.



Java™ Education
and Technology Services



Invest In Yourself,
Develop Your Career

Agenda

- A. The History and Evolution of Java
- B. An Overview of Java
- C. Data Types, Variables and Arrays
- D. Operators, Control Statements and String Handling
- E. Modifiers, Access Specifiers, Packages and Interfaces
- F. Wrapper Classes, Autoboxing and Inner Classes
- G. Exception Handling
- H. Generics and Lambda Expressions
- I. Java Stream API
- J. Java Collections
- K. Multi-Threading in Java

Lesson 8

Generics and Lambda Expressions and Method Reference



Lambda Expressions

Introduction



Lambda Expressions

- ▶ Lambda expressions could be considered as the most exciting change to the Java language in many years.
- ▶ *A lambda expression is a block of code that you can pass around so it can be executed later, once or multiple times.*

Lambda Expressions

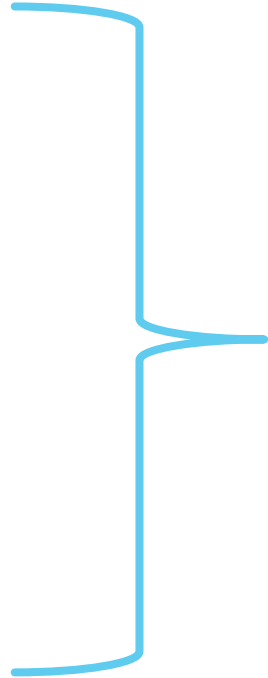
- ▶ Java defines two types of lambda bodies.
 - ▶ single expression lambda bodies
 - ▶ block of code lambda bodies

lambda expression λ .

Single Lambda Expressions(Examples)

let's look at some simple examples that put the basic lambda expression concepts into action

```
// A functional interface.  
interface MyValue  
{  
    double getValue();  
}  
  
// Another functional interface.  
interface MyParamValue  
{  
    double getValue(double v);  
}
```



Functional Interfaces

Single Lambda Expressions(Examples)

```
public class LambdaDemo {  
  
    public static void main(String[] args) {  
  
        MyValue myVal = new MyValue(){double getValue() {  
            return 98.6;  
        }  
    };  
  
    System.out.println("A constant value: " + myVal.getValue());  
}
```


Single Lambda Expressions(Examples)

```
public class LambdaDemo {  
  
    public static void main(String[] args) {  
  
        MyValue myVal;  
  
        myVal = () -> 98.6;  
  
        System.out.println("A constant value: " + myVal.getValue());  
    }  
}
```



A simple lambda expression

Single Lambda Expressions(Examples)

```
MyParamValue myPval = new MyParamValue(){  
    double getValue(double n){  
        return 1.0 / n  
    }  
  
};  
  
// Call getValue(v) through the myPval reference.  
System.out.println("Reciprocal of 4 is " + myPval.getValue(4.0));  
System.out.println("Reciprocal of 8 is " + myPval.getValue(8.0));  
}  
}
```

Sample output from the program is shown here:
Reciprocal of 4 is 0.25
Reciprocal of 8 is 0.125

Single Lambda Expressions(Examples)

```
/*
 * Now, create a parameterized lambda expression and assign it to a MyParamValue
 * reference. This lambda expression returns the reciprocal of its argument.
 */
```

```
MyParamValue myPval = (n) -> 1.0 / n;
```



A lambda expression that
has a parameter

```
// Call getValue(v) through the myPval reference.
```

```
System.out.println("Reciprocal of 4 is " + myPval.getValue(4.0));
```

```
System.out.println("Reciprocal of 8 is " + myPval.getValue(8.0));
```

```
}
}
```

Sample output from the program is shown here:

```
Reciprocal of 4 is 0.25
Reciprocal of 8 is 0.125
```

Single Lambda Expressions(Examples)

```
// Use the same functional interface with three  
//different lambda expressions.
```

```
// A functional interface that takes two int  
//parameters and returns  
// a boolean result.
```

```
interface NumericTest {  
boolean test(int n, int m);  
}
```

Single Lambda Expressions(Examples)

```
class LambdaDemo2 {  
    public static void main(String args[]) {  
        // This lambda expression determines if one number  
        // is //factor of another.  
  
        NumericTest isFactor = (n, d) -> (n % d) == 0;  
  
        if(isFactor.test(10, 2))  
            System.out.println("2 is a factor of 10");  
    }  
}
```

Single Lambda Expressions(Examples)

```
if(!isFactor.test(10, 3))  
    System.out.println("3 is not a factor of 10");  
    System.out.println();  
  
// This lambda expression returns true if the  
// first //argument is less than the second.  
NumericTest lessThan = (n, m) -> (n < m);  
if(lessThan.test(2, 10))  
    System.out.println("2 is less than 10");
```

Single Lambda Expressions(Examples)

```
if(!lessThan.test(10, 2))  
    System.out.println("10 is not less than 2");  
System.out.println();  
  
// This lambda expression returns true if the absolute  
// values of the arguments are equal.  
NumericTest absEqual = (n, m) -> (n < 0 ? -n : n) == (m < 0 ? -m : m);
```

Single Lambda Expressions(Examples)

```
if(absolute.test(4, -4))  
    System.out.println("Absolute values of 4 and -4 are equal.");  
  
if(!lessThan.test(4, -5))  
    System.out.println("Absolute values of 4 and -5 are not  
equal.");  
System.out.println();  
}  
}
```


Single Lambda Expressions(Examples)

The output is shown here:

2 is a factor of 10

3 is not a factor of 10

2 is less than 10

10 is not less than 2

Absolute values of 4 and -4 are equal.

Absolute values of 4 and -5 are not equal.

Block Lambda Expressions

Lambda Syntax

(parameters) -> expression

or

() -> expression

or

(parameters) -> { statements; }



Single Lambda
Expressions



Block Lambda
Expressions

Block Lambda Expressions

- ▶ Lambdas that have block bodies are sometimes referred to as *block lambdas*.
- ▶ A block lambda expands the types of operations that can be handled within a lambda expression because it allows the body of the lambda to contain multiple statements.
- ▶ Aside from allowing multiple statements, block lambdas are used much like the expression lambdas just discussed.
- ▶ One key difference, however, is that you must explicitly use a **return** statement to return a value.

Block Lambda Expressions

- ▶ Here is an example that uses a block lambda to find the smallest positive factor of an `int` value.
- ▶ It uses an interface called **NumericFunc** that has a method called `func( )`, which takes one `int` argument and returns an `int` result. Thus, **NumericFunc** supports a numeric function on values of type `int`.

```
// A block lambda that finds the smallest positive factor  
// of an int value.  
interface NumericFunc {  
    int func(int n);  
}
```

Block Lambda Expressions

```
class BlockLambdaDemo {  
    public static void main(String args[]) {  
        // This block lambda returns the smallest positive factor of a value.  
        NumericFunc smallestF = (n) -> {  
            int result = 1;  
            // Get absolute value of n.  
            n = n < 0 ? -n : n;  
            for(int i=2; i <= n/i; i++)  
                if((n % i) == 0) {  
                    result = i;  
                    break;  
                }  
            return result;  
        };  
        System.out.println("Smallest factor of 12 is " + smallestF.func(12));  
        System.out.println("Smallest factor of 11 is " + smallestF.func(11)); } }
```

Block Lambda Expression

Generic Functional Interface

- ▶ The functional interface associated with a lambda expression can be generic.

```
// Use a generic functional interface.  
  
// A generic functional interface with two parameters  
// that returns a boolean result.  
  
interface SomeTest<T>  
{  
  
    boolean test(T n, T m);  
  
}
```

Generic Functional Interface

```
class GenericFunctionalInterfaceDemo {  
    public static void main(String args[]) {  
        // This lambda expression determines if one integer is a factor of another.  
        SomeTest<Integer> isFactor = (n, d) -> (n % d) == 0;  
        if(isFactor.test(10, 2)) System.out.println("2 is a factor of 10");  
        System.out.println();  
        // The next lambda expression determines if one Double is a factor of another.  
        SomeTest<Double> isFactorD = (n, d) -> (n % d) == 0;  
        if(isFactorD.test(212.0, 4.0)) System.out.println("4.0 is a factor of 212.0");  
        System.out.println();  
    }  
}
```

Generic Functional Interface

```
// This lambda expression determines if one string is part of another.  
SomeTest<String> isIn = (a, b) -> a.indexOf(b) != -1;  
  
String str = "Generic Functional Interface";  
System.out.println("Testing string: " + str);  
  
if(isIn.test(str, "face")) System.out.println("'face' is found.");  
else System.out.println("'face' not found.");  
}  
}
```

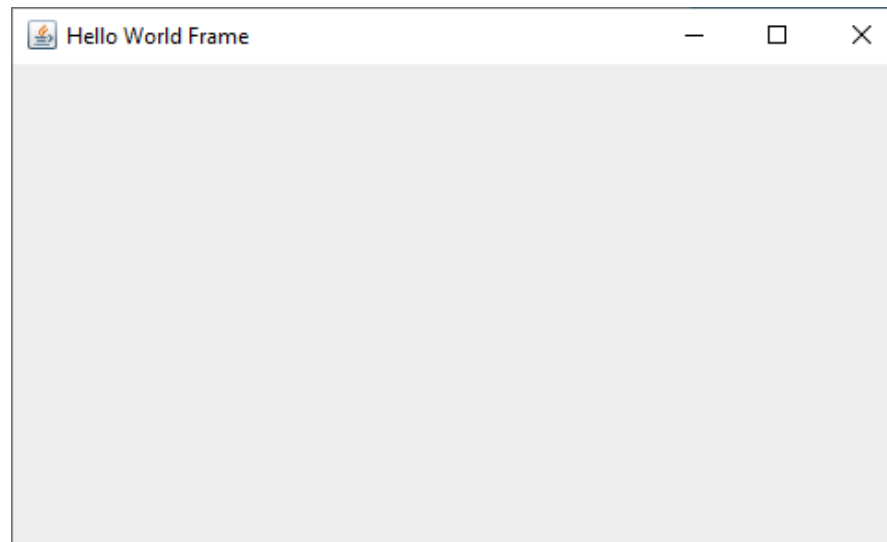

Lesson 11

Multi-Threading in Java



Create Simple GUI - java Swing Frame

- ▶ **JFrame** is a class of the **javax.swing** package .
- ▶ This is the top-level window, with border and a title bar.
- ▶ JFrame class has various methods which can be used to customize it.



Create Simple GUI - java Swing Frame

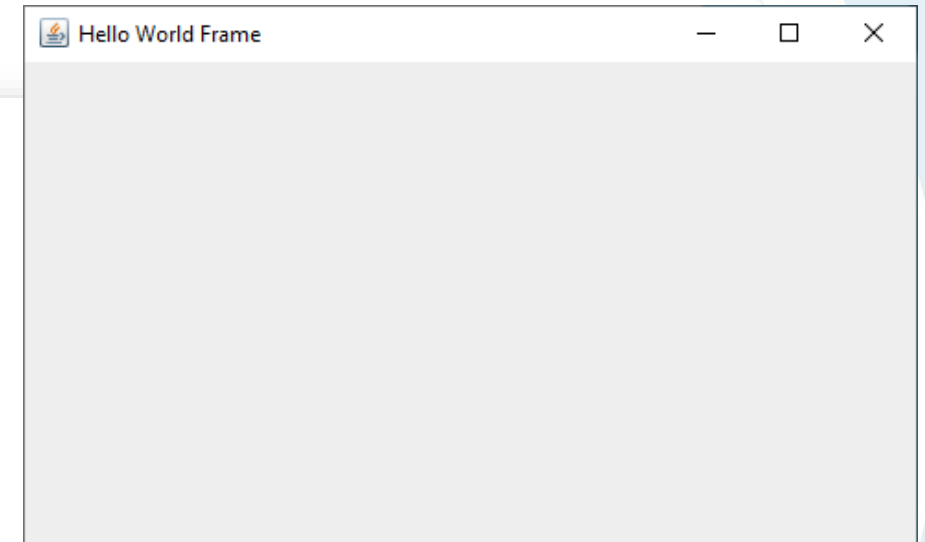
```
package thrdPkg;

import javax.swing.JFrame;

public class SimpleFrame {

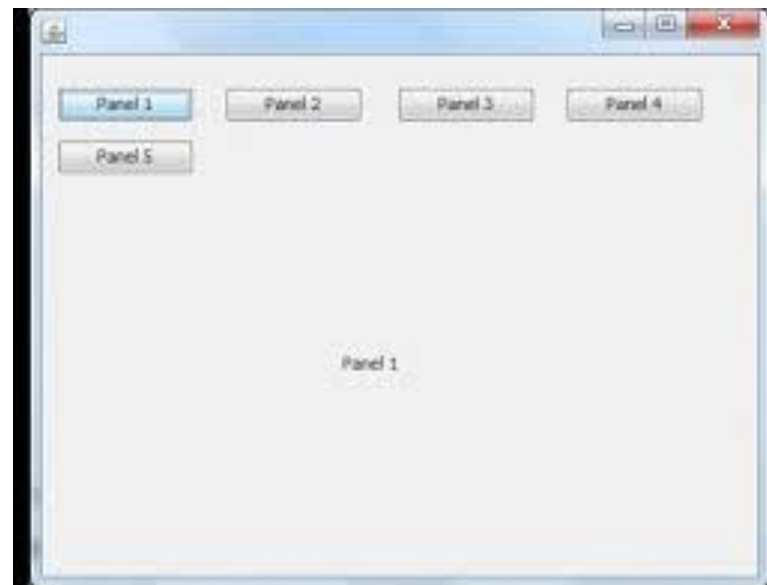
    public static void main(String[] argv) {
        // Create a frame
        JFrame f = new JFrame();

        // Set the title and other parameters.
        f.setTitle("Hello World Frame");
        f.setSize(500, 300);
        f.setVisible(true);
    }
}
```



Create Simple GUI - java Swing Panel

- ▶ **JPanel**, a part of the Java Swing package, is a container that can store a group of components.
- ▶ The main task of JPanel is to organize components, various layouts can be set in JPanel which provide better organization of components.



Create Simple GUI - java Swing Panel

```
import java.awt.Color;
import java.awt.Graphics;
import java.util.Date;
import javax.swing.JPanel;
```

```
class MyPanel extends JPanel {
```

```
    public MyPanel() {
        this.setBackground(Color.cyan);
    }
```

@Override

```
    public void paintComponent(Graphics g) {
        // Always call super.paintComponent (g):
        super.paintComponent(g);
        g.drawString(new Date().toLocaleString(), 100, 100);
    }
}
```

1

Feb 22, 2022 2:21:54 PM

Create Simple GUI

2

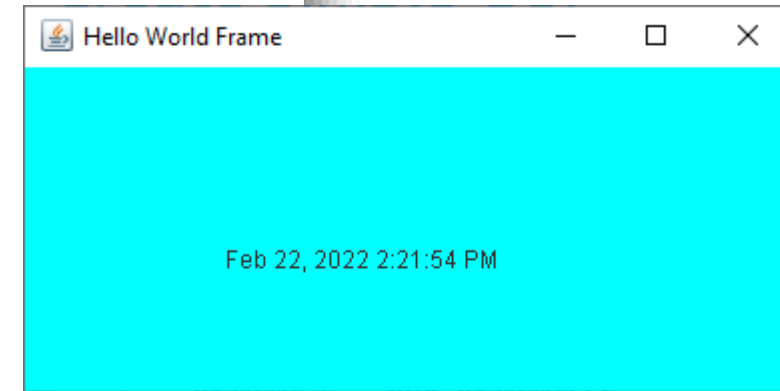
```
package thrdPkg;

import javax.swing.JFrame;

public class SimpleFrame {

    public static void main(String[] argv) {
        // Create a frame
        JFrame f = new JFrame();

        // Set the title and other parameters.
        f.setTitle("Hello World Frame");
        MyPanel mp=new MyPanel();
        f.setContentPane(mp);
        f.setSize(400, 200);
        f.setVisible(true);
    }
}
```





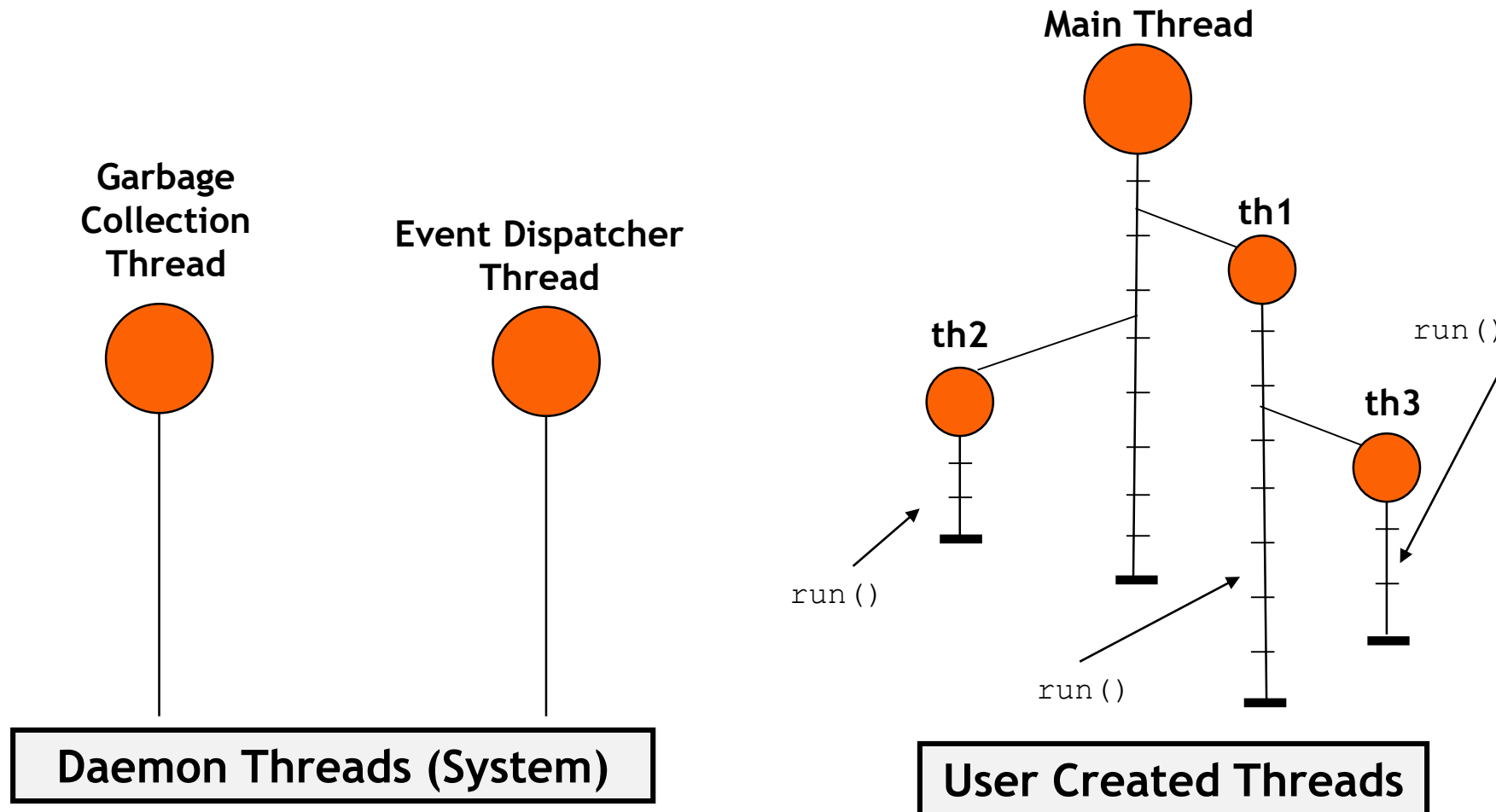
Multi- Threading

What is Thread?

- ▶ A Thread is
 - ▶ A single sequential execution path in a program
 - ▶ Used when we need to execute two or more program segments concurrently (multitasking).
 - ▶ Used in many applications:
 - ▶ Games, animation, perform I/O
 - ▶ Every program has at least two threads.
 - ▶ Each thread has its own stack, priority & virtual set of registers.
- ▶ Multiple threads does not mean that they execute in parallel when you're working in a single CPU.
 - ▶ Some kind of scheduling algorithm is used to manage the threads (e.g. Round Robin).
 - ▶ The scheduling algorithm is JVM specific (i.e. depending on the scheduling algorithm of the underlying operating system)

Threads

- ▶ several thread objects that are executing concurrently:



Threads

- ▶ Threads that are ready for execution are put in the ready queue.
 - ▶ Only one thread is executing at a time, while the others are waiting for their turn.
- ▶ The task that the thread carries out is written inside the `run ()` method.

The Thread Class

- **Class Thread**

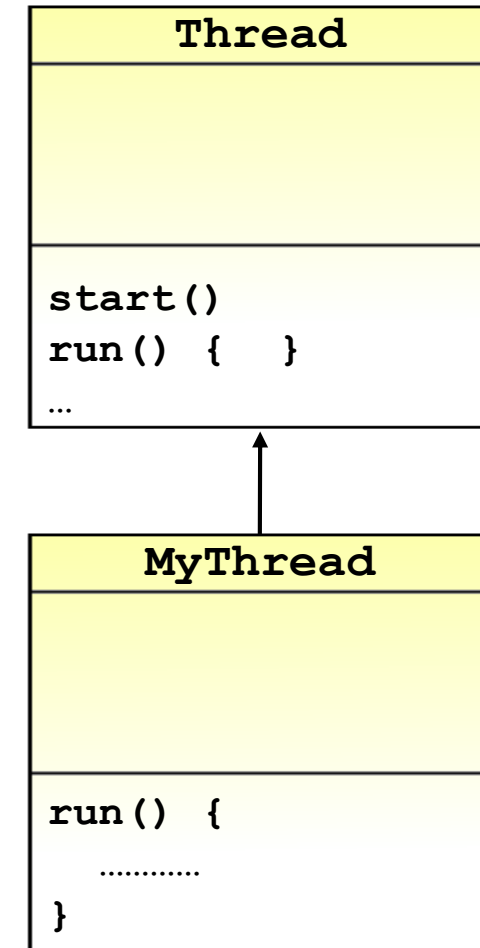
- `start()`
- `run()`
- `sleep() *`
- `suspend() *`
- `resume() *`
- `stop() *`

Working with Threads

► There are two ways to work with threads:

- **Extending Class Thread:**

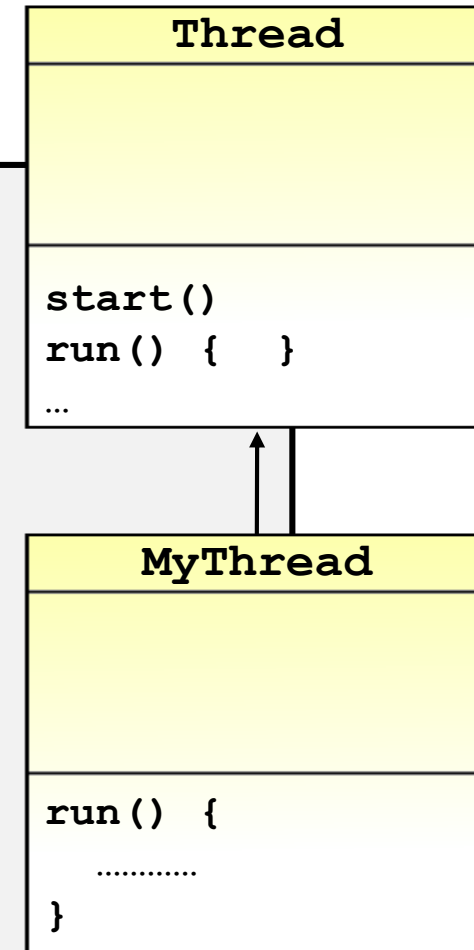
1. Define a class that extends **Thread**.
2. Override its **run ()** method.
3. In main or any other method:
 - a. Create an object of the subclass.
 - b. Call method **start ()**.



Working with Threads

```
public class MyThread extends Thread
{
    public void run()
    {
        • in main() or any method:
        ... //write the job here
    }
}
```

```
public void anyMethod()
{
    MyThread th = new MyThread();
    th.start();
}
```



Example:

```
public class SimpleThread extends Thread {
    String name;
    public SimpleThread(String name) { ...3 lines }

    @Override
    public void run() {
        for (int i = 0; i < 10; i++) {
            System.out.println(i + " " + name);
            try {
                sleep((int) (Math.random() * 1000));
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        System.out.println("DONE! " + name);
    }
}
```

```
0 Thread Object 1
1 Thread Object 1
2 Thread Object 1
3 Thread Object 1
4 Thread Object 1
5 Thread Object 1
6 Thread Object 1
7 Thread Object 1
8 Thread Object 1
9 Thread Object 1
DONE! Thread Object 1
```

```
public class Lecture_Demo {

    public static void main(String[] args) {

        new SimpleThread("Thread Object 1").start();
    }
}
```

Example:

```
public class SimpleThread extends Thread {
    String name;
    public SimpleThread(String name) { ...3 lines }

    @Override
    public void run() {
        for (int i = 0; i < 10; i++) {
            System.out.println(i + " " + name);
            try {
                sleep((int) (Math.random() * 1000));
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        System.out.println("DONE! " + name);
    }
}
```

```
0 Thread Object 1
0 Thread object 2
1 Thread object 2
2 Thread object 2
1 Thread Object 1
3 Thread object 2
4 Thread object 2
2 Thread Object 1
3 Thread Object 1
5 Thread object 2
6 Thread object 2
4 Thread Object 1
7 Thread object 2
5 Thread Object 1
8 Thread object 2
6 Thread Object 1
9 Thread object 2
DONE! Thread object 2
7 Thread Object 1
8 Thread Object 1
9 Thread Object 1
DONE! Thread Object 1
```

```
public class Lecture_Demo {

    public static void main(String[] args) {

        new SimpleThread("Thread Object 1").start();
        new SimpleThread("Thread object 2").start();
    }
}
```

Working with Threads

► There are two ways to work with threads:

- **Implementing Interface *Runnable*:**

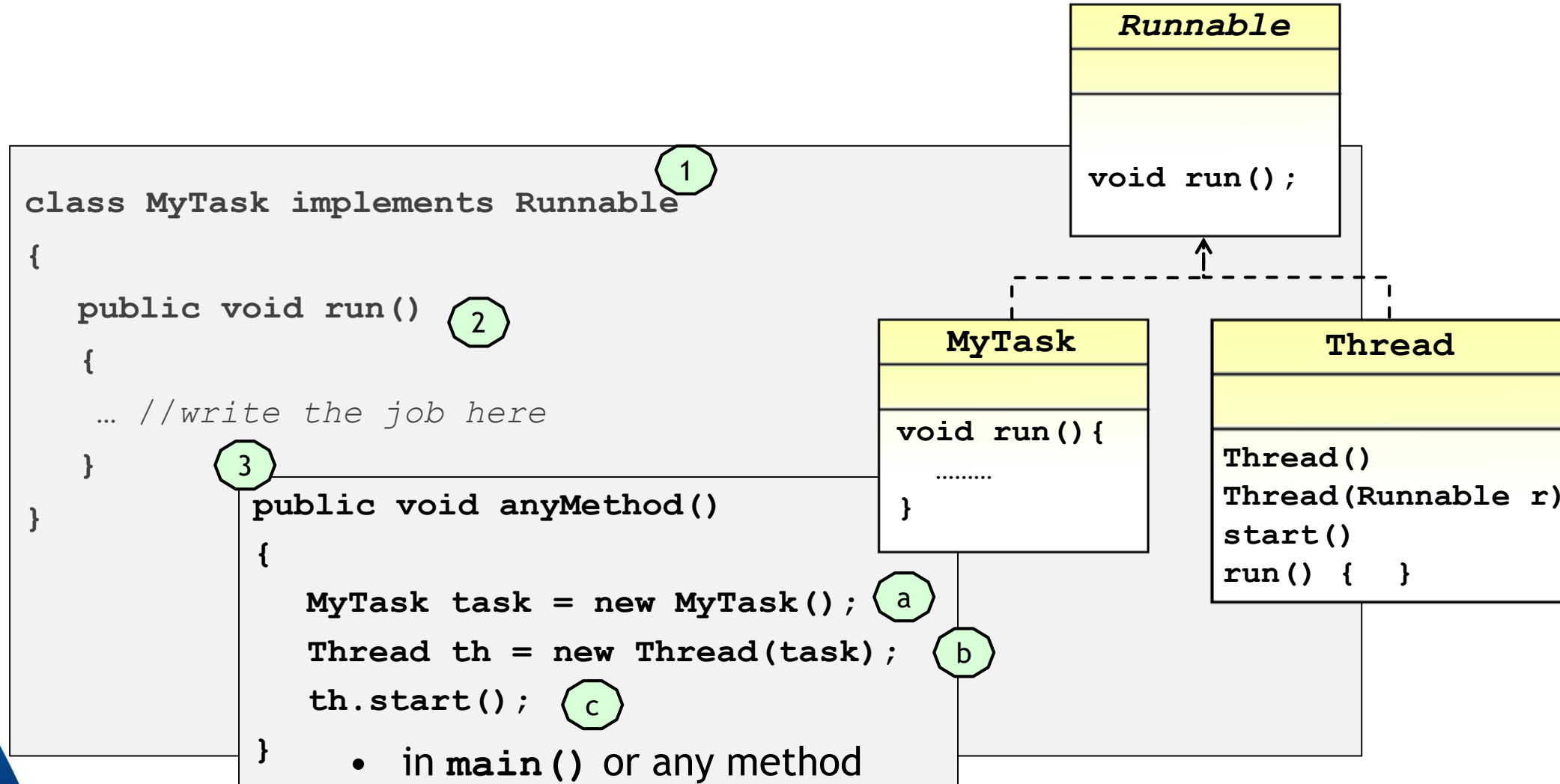
1. Define a class that implements **Runnable**.

2. Override its **run ()** method .

3. In main or any other method:

- a. Create an object of your class.
- b. Create an object of class **Thread** by passing your object to the constructor that requires a parameter of type **Runnable**.
- c. Call method **start ()** on the **Thread** object.

Working with Threads



Extending Thread VS. Implementing Runnable

- ▶ Choosing between these two is a matter of taste.
- ▶ Implementing the Runnable interface:
 - ▶ May take more work since we still:
 - ▶ Declare a Thread object
 - ▶ Call the Thread methods on this object
 - ▶ Your class can still extend other class
- ▶ Extending the Thread class
 - ▶ Easier to implement
 - ▶ Your class can no longer extend any other class

Example:

```
public class SimpleRunThread implements Runnable{
    String name;

    public SimpleRunThread(String name) {
        this.name = name;
    }

    @Override
    public void run() {
        for (int i = 0; i < 10; i++) {
            System.out.println(i + " " + name);
            try {
                sleep((int) (Math.random() * 1000));
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        System.out.println("DONE! ");
    }
}
```

```
public static void main(String[] args) {
    new SimpleThread("Thread Object 1").start();
    new SimpleThread("Thread object 2").start();

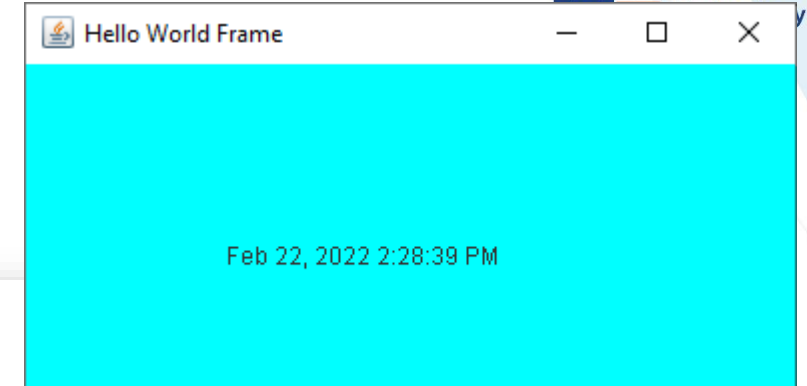
    new Thread(new SimpleRunThread("runnable object")).start();
}
```

```
5 Thread object 2
6 Thread object 2
4 Thread Object 1
4 runnable object
7 Thread object 2
5 Thread Object 1
5 runnable object
6 Thread Object 1
7 Thread Object 1
6 runnable object
8 Thread object 2
8 Thread Object 1
9 Thread Object 1
9 Thread object 2
```

Create Simple GUI - java Swing Panel with Thread

```
class MyPanel extends JPanel implements Runnable{

    public MyPanel() {
        this.setBackground(Color.cyan);
        new Thread(this).start();
    }
    @Override
    public void paintComponent(Graphics g) { ...5 lines }
    @Override
    public void run() {
        while(true){
            try {
                this.repaint();
                Thread.sleep(1000);
            } catch (InterruptedException ex) {
                Logger.getLogger(MyPanel.class.getName()).log(Level.SEVERE, null, ex);
            }
        }
    }
}
```

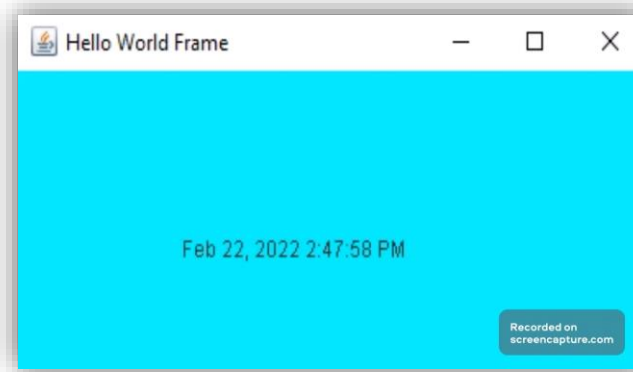




Lab Exercises

Lab Exercise 1

- ▶ Simple Date and Time JFrame Application.



Lab Exercise 2

- ▶ Make a text **marquee** (is a scrolling piece of **text** displayed horizontally across your App).



Lab Exercise 3

- ▶ Make a Bouncing Ball App.

